OnDeviceTraining
Code Reading Guide 2

Line-by-Line Syntax & Explanation — Beginner Edition

Mohamed · Master's Thesis · 2026

This guide walks through every key source file in the codebase with actual C code, a syntax label for every line, and a plain-English explanation written for readers who are new to C programming. Every abbreviation is written in full. Read Guide 1 first to understand the overall phase structure; use this guide to understand the code itself.

Phase 1 — Foundation

Common.h · Rounding.h / Rounding.c · DTypes.h / DTypes.c

1 · Common.h

`src/common/include/Common.h`

Common.h is a **header-only file** — it contains no functions, only preprocessor macros. It is included by *every other .c file* in the project. It provides three debug-print macros (for printing coloured messages at different severity levels) and one raw byte-dump macro that prints the contents of a memory buffer in hexadecimal. Everything here is controlled by a **compile-time verbosity level** called DLEVEL (Debug Level), so all of this output can be completely removed from the final firmware by not passing any debug flag to the compiler. Nothing in this file runs at program start-up; macros are text replacements that happen before the compiler sees the code.

`#ifndef ODT_COMMON_H`	**Include guard — top half.** "#ifndef" means "if not defined". ODT_COMMON_H is a unique symbol (ODT stands for OnDeviceTraining). If this symbol has not been defined yet, the compiler will process everything between this line and the matching #endif at the bottom. If the symbol IS already defined (meaning this file was already included once), the compiler skips all the way to #endif and does nothing. This prevents errors that would occur if the same type definitions or macros were declared twice.
`#define ODT_COMMON_H`	**Define the guard symbol.** Now that the file is being processed for the first time, we immediately define the guard symbol. Any future #include of Common.h will see this symbol is defined and skip the file entirely.
`#include <stdbool.h>`	**Pull in the boolean type.** The C99 standard header stdbool.h defines the keyword "bool" and the values "true" and "false". These are used by the do{}while(false) idiom inside the macros — without this header, the word "false" would not be recognised by the compiler.
`#include <stdio.h>`	**Pull in the standard input/output library.** stdio.h provides the printf() function, which is the underlying output mechanism used by all the debug macros. Without this include, printf would be an undeclared function and the compiler would report an error.
`#include <string.h>`	**Pull in string utility functions.** string.h is not used directly in this file, but callers that include Common.h often need functions like memcpy() or memset(), which are declared in string.h. Including it here saves every individual file from having to include it separately.
`#ifndef SOURCE_FILE`	**Fallback guard for the source file label.** Each .c file in the project is expected to define SOURCE_FILE (e.g. #define SOURCE_FILE "ROUNDING") before including Common.h. This #ifndef checks whether the .c file forgot to do so, and if so provides a default warning string.
`#define SOURCE_FILE "no Source file defined!"`	**Default label string.** If a .c file omits its own #define SOURCE_FILE, the debug macros will print this fallback text instead of crashing or printing garbage. It signals to the developer that they need to add the define to their file.
`#endif`	**Closes the SOURCE_FILE fallback guard.** Every #ifndef or #ifdef must have a matching #endif. This one ends the "did the file forget its SOURCE_FILE?" check.

`#ifdef DEBUG_MODE_DEBUG`	**Check for the most verbose debug flag.** If the compiler was invoked with the flag -DDEBUG_MODE_DEBUG (e.g. "gcc -DDEBUG_MODE_DEBUG ..."), this condition is true and DLEVEL is set to 3 — the highest verbosity level, which enables all three categories of messages.
`#define DLEVEL 3`	**Set debug level to 3 (most verbose).** DLEVEL 3 means the program will print DEBUG messages (fine-grained tracing), INFO messages (progress summaries), and ERROR messages (failures). The number 3 is just a constant — higher means more output.
`#elif defined(DEBUG_MODE_INFO)`	**Else-if chain — check for medium verbosity.** #elif means "else if". If DEBUG_MODE_DEBUG was not set but DEBUG_MODE_INFO was, we land here and set DLEVEL to 2.
`#define DLEVEL 2`	**Set debug level to 2 (medium verbosity).** Only INFO and ERROR messages will print. Fine-grained DEBUG traces are suppressed.
`#elif defined(DEBUG_MODE_ERROR)`	**Check for the least verbose flag.** Only error messages will be shown.
`#define DLEVEL 1`	**Set debug level to 1 (errors only).** The program prints only when something goes wrong.
`#else`	**Default case — no debug flag was passed.** This branch runs when none of the three debug symbols were defined on the compiler command line.
`#define DLEVEL 0`	**Set debug level to 0 (completely silent).** All debug, info, and error print calls compile away to nothing. This is the default for firmware builds on the Microcontroller Unit, where printing is expensive and the output has nowhere to go.
`#endif`	**Closes the debug-level selection ladder.** The compiler has now chosen exactly one value for DLEVEL.

The three debug macros all use the same pattern — a **do{}while(false)** block. This pattern is explained below with the first macro:

`#define PRINT_DEBUG(str, ...) \`	**Begin the PRINT_DEBUG macro definition.** The backslash at the end of the line tells the preprocessor "this definition continues on the next line". The "..." in the parameter list means the macro accepts a variable number of extra arguments, just like printf() does. For example you can call PRINT_DEBUG("value = %d", myInt).
`do { \`	**Open a do-while block.** The do{}while(false) idiom is a standard C trick for writing multi-statement macros safely. Without it, using the macro after an if-statement without curly braces would cause confusing bugs. The do keyword starts the block.
`if (DLEVEL >= 3) { \`	**Runtime verbosity check.** Even though DLEVEL is set at compile time, this if-statement checks it at runtime inside the macro. The compiler is smart enough to see that DLEVEL is a constant and will remove the entire block from the binary when DLEVEL < 3. This means zero overhead in release builds.
`printf("\033[0;33m[%s: %s] ", SOURCE_FILE, __FUNCTION__); \`	**Print a coloured prefix.** The sequence \033[0;33m is an ANSI terminal escape code that switches the terminal text colour to yellow. %s is a format specifier for a string. SOURCE_FILE is the file name defined in each .c file. __FUNCTION__ is a predefined C identifier that the compiler automatically replaces with the name of the function currently being compiled — this tells you exactly where the debug message came from.

`printf(str, ##__VA_ARGS__); \`	**Print the user's message.** "str" is the format string the caller passes, and ##__VA_ARGS__ expands to any additional arguments. The ## (double hash) is a GNU extension: if no extra arguments are given, it removes the comma that would otherwise appear before the empty argument list, preventing a syntax error.
`printf("\033[0m\n"); \`	**Reset colour and print a newline.** \033[0m is the ANSI escape code to reset all terminal formatting back to normal. \n moves the cursor to the next line. Without this, subsequent terminal output would still appear in yellow.
`} \`	**Close the if block.**
`} while (false)`	**Close the do-while loop.** The condition "false" means the loop body runs exactly once and then exits — it is never a real loop. The purpose is purely syntactic: it makes the macro expand to a single statement that requires a semicolon after it, which matches how normal function calls look and behaves correctly in all contexts.

PRINT_INFO is identical to PRINT_DEBUG but uses DLEVEL >= 2 and prints without colour codes. PRINT_ERROR uses DLEVEL >= 1 and prints in red (ANSI escape code \033[0;31m).

`#define PRINT_BYTE_ARRAY(prefix, byteArray, numberOfBytes) \`	**Macro to dump the raw contents of a memory buffer.** This does not check DLEVEL — it always prints. It is used for one-off inspection of binary data such as serialized tensors. "prefix" is a label string the caller provides. "byteArray" is a pointer to the memory to inspect. "numberOfBytes" is how many bytes to print.
`do { \`	**Same do{}while(false) safety wrapper.**
`printf("[%s: %s] ", SOURCE_FILE, __FUNCTION__); \`	**Print file and function name prefix** — same as the debug macros but without colour codes.
`printf(prefix); \`	**Print the caller-supplied label string** — e.g. "weights: " or "gradient buffer: " — so the output is self-documenting.
`for (size_t index = 0; index < numberOfBytes; index++) { \`	**Loop over every byte in the buffer.** size_t is the unsigned integer type used for sizes and counts in C. It is guaranteed to be large enough to hold any object size on the current platform.
`printf("0x%02X ", byteArray[index]); \`	**Print each byte in hexadecimal.** "0x" is the conventional prefix for hexadecimal numbers. %02X means: print as uppercase hexadecimal, with at least 2 digits, padding with a leading zero if necessary (so 5 prints as 05, not just 5). This makes the output easy to read and compare against memory dump tools.
`printf("\n"); \`	**Print a newline** after all the bytes have been printed.
`} while (false)`	**End of the macro.**

2 · Rounding.h / Rounding.c

`src/arithmetic/include/Rounding.h + src/arithmetic/Rounding.c`

Rounding.c provides the two rounding strategies used throughout quantization. The first is **Half-To-Even** rounding (also called banker's rounding), which is the standard deterministic strategy: 0.5 rounds to the nearest even integer. The second is **Stochastic Rounding Half-To-Even**, which is the core mathematical trick that makes Fully Quantized Training work. Stochastic Rounding adds a small random value (called a *dither*) before rounding, so that a gradient value of 0.1 will round up to 1 about 10% of the time and round down to 0 about 90% of the time — preserving the *expected value* of the gradient even though individual samples are rounded to integers.

Header — Rounding.h:

`#include <stdint.h>`	**Pull in fixed-width integer types.** stdint.h defines types like int32_t (a signed 32-bit integer), uint8_t (an unsigned 8-bit integer), etc. These types have guaranteed sizes on every platform, unlike plain "int" which can vary.
`typedef enum roundingMode {`	**Begin defining a named enumeration type.** An enum (enumeration) is a way to define a set of named integer constants. "typedef" means we are also creating a type alias so we can write "roundingMode_t" instead of the longer "enum roundingMode" everywhere.
`HTE,`	**First enumerator — Half-To-Even.** The compiler assigns this the integer value 0 automatically. Half-To-Even is the standard deterministic rounding rule: when a value is exactly halfway between two integers (like 2.5), it rounds to whichever integer is even (2 in this case). This is the same rule used by most programming languages by default.
`SRHTE`	**Second enumerator — Stochastic Rounding Half-To-Even.** The compiler assigns this integer value 1. Stochastic Rounding adds a random number uniformly distributed between -0.5 and +0.5 to the input before applying Half-To-Even rounding. This means the result is random, but its statistical average equals the original float value — an important property for training neural networks with integers.
`} roundingMode_t;`	**Close the enum and create the type alias.** After this line, "roundingMode_t" is a valid type name. Variables of this type can hold either HTE (value 0) or SRHTE (value 1).
`int32_t roundByMode(float input, roundingMode_t roundingMode);`	**Function declaration.** This tells the compiler that a function called roundByMode exists somewhere, takes a 32-bit floating-point number and a rounding mode, and returns a 32-bit signed integer. The actual code (the function body) is in Rounding.c.
`float clamp(float input, float min, float max);`	**Declaration of the saturation helper.** "Clamping" means forcing a value to stay within a given range. If the input is below min, return min. If it is above max, return max. Otherwise return the input unchanged. This prevents integer overflow during quantization.

Implementation — Rounding.c:

`#define SOURCE_FILE "ROUNDING"`	**Set this file's label for debug messages.** Any call to PRINT_DEBUG or PRINT_ERROR inside this file will include the text "ROUNDING" in the output, making it easy to trace which module produced a given debug line.
`#include <math.h>`	**Pull in mathematical functions.** math.h provides round(), which is the C standard library function for rounding a float to the nearest integer. Without this include, the compiler would not know what round() is.
`#include "Rounding.h"`	**Include own header.** The .c file includes its own .h file to get access to the roundingMode_t type definition it declared there. Quoted include paths ("...") search the local project directories first, unlike angle-bracket includes (<...>) which search system directories.
`#include "RNG.h"`	**Include the Random Number Generator module.** Stochastic Rounding Half-To-Even needs to generate a random floating-point number for the dither. The function rngNextFloat() is declared in RNG.h and provides this.
`int32_t roundHTE(float input) {`	**Internal Half-To-Even rounding function.** This function is not declared in the header, so it is only usable inside Rounding.c (it is "file-private"). It wraps the C standard library round() function.

`return round(input);`	**Delegate to the C standard library.** The C99 function round() ties away from zero (2.5 → 3, -2.5 → -3), which is close enough to banker's rounding for all non-exact-half cases and simpler to implement correctly.
`float randfloat() {`	**Thin wrapper around rngNextFloat().** This helper exists so that Rounding.c does not need to know the internal details of the Random Number Generator module. If the random number generator is ever replaced, only this one line needs updating.
`return rngNextFloat();`	**Returns a random float in the range [0, 1).** The square bracket [means 0 is included; the round bracket) means 1 is excluded. The Random Number Generator is a fast Exclusive-OR shift algorithm (explained in section 11).
`int32_t roundSRHTE(const float input) {`	**Stochastic Rounding Half-To-Even implementation.** "const" means the function promises not to modify the input. The function adds a random dither before rounding.
`return roundHTE(input + randfloat() - 0.5f);`	**The dither-and-round operation.** randfloat() returns a number in [0, 1). Subtracting 0.5 shifts it to the range [-0.5, 0.5). Adding this to the input shifts it by a random amount, then Half-To-Even rounding snaps it to an integer. A value of 0.1 will be shifted to somewhere in [-0.4, 0.6). About 10% of the time (when the shift pushes it above 0.5) it rounds up to 1, and 90% of the time it rounds down to 0. Over many training steps, the average is preserved at 0.1.
`int32_t roundByMode(const float input, const roundingMode_t roundingMode) {`	**The public dispatch function.** This is the only rounding function that other modules call. It receives a float and a mode enum value, and calls the correct internal function. Both parameters are "const" to prevent accidental modification inside the function.
`switch (roundingMode) {`	**Switch statement on the enum value.** A switch is like a multi-way if-else but more efficient. The compiler can sometimes turn it into a direct jump table (a lookup table of instruction addresses) rather than a chain of comparisons.
`case HTE: return roundHTE(input);`	**Deterministic path.** If the mode is Half-To-Even, call the deterministic rounding function. This path is used during inference and anywhere predictable behavior is required.
`case SRHTE: return roundSRHTE(input);`	**Stochastic path.** If the mode is Stochastic Rounding Half-To-Even, call the randomized rounding function. This path is used during the backward pass of Fully Quantized Training.
`return 0;`	**Unreachable safety return.** If roundingMode has some unexpected integer value not covered by the switch cases, this line returns 0 instead of undefined behavior. More importantly, it suppresses a compiler warning about "control reaching end of non-void function".
`float clamp(float input, float min, float max) {`	**Saturation (clamping) function.** Takes three floats and returns the input forced into the range [min, max].
`if (input < min) { return min; }`	**Lower bound enforcement.** If the input is less than the minimum allowed value, return the minimum. In quantization, this prevents values from wrapping around (overflow) when converted to a smaller integer type.
`if (input > max) { return max; }`	**Upper bound enforcement.** Same logic for the maximum.
`return input;`	**Pass-through case.** The value is already within the valid range, so return it unchanged.

Why Stochastic Rounding Half-To-Even matters for training: In standard 8-bit integer training, any gradient value smaller than half a Least Significant Bit always rounds to zero and permanently vanishes. With Stochastic Rounding, even a tiny gradient of 0.001 will round to 1 about 0.1% of the time instead of always becoming 0, preserving its expected value. This allows the network to learn from very small gradient signals that deterministic rounding would destroy — this is the central insight of Fully Quantized Training.

3 · DTypes.h / DTypes.c

`src/tensor/include/DTypes.h + src/tensor/DTypes.c`

DTypes provides portable read/write helper functions for 32-bit signed integer and 32-bit floating-point values stored inside raw byte arrays (arrays of type uint8_t, meaning unsigned 8-bit integer). The central technique is **memcpy-based type punning** — the only portable and well-defined way in C to reinterpret the bit pattern of a float as an integer, or vice versa. The alternative (casting a float pointer to an int pointer and dereferencing it) violates C's strict aliasing rule and causes undefined behaviour on modern compilers.

`int32_t readBytesAsInt32(uint8_t *bytes) {`	**Read 4 consecutive bytes and interpret them as a signed 32-bit integer.** "uint8_t *bytes" is a pointer to the start of the byte buffer. uint8_t is an unsigned 8-bit integer — the smallest addressable unit of memory. A 32-bit integer occupies exactly 4 bytes, so this function reads 4 bytes.
`int32_t x;`	**Declare a local variable to receive the result.** int32_t is a signed 32-bit integer. It is declared but not yet given a value.
`memcpy(&x, bytes, sizeof(int32_t));`	**The type-punning step.** memcpy copies raw bytes from one location to another. Here it copies sizeof(int32_t) bytes — which is always 4 — from the input byte array into the memory of the local variable x. After this, x holds the exact bit pattern from the 4 bytes, reinterpreted as an integer. sizeof() is a compile-time operator that returns the size of a type in bytes.
`return x;`	**Return the reconstructed integer.**
`float readBytesAsFloat(uint8_t *bytes) {`	**Same pattern for 32-bit floating-point values.** A float also occupies 4 bytes on all platforms targeted by this project.
`float x;`	**Local float variable.**
`memcpy(&x, bytes, sizeof(float));`	**Byte-reinterpretation via memcpy.** This is the only C-standard-conforming way to treat 4 bytes of memory as a float without invoking undefined behaviour. A direct cast like *(float*)bytes would technically be undefined behaviour under strict aliasing rules, even though it would "work" on most compilers.
`return x;`	**Return the reconstructed float.**
`void readBytesAsInt32Array(size_t numberOfValues, uint8_t *bytes, int32_t *outputArray) {`	**Batch version — read n consecutive 32-bit integers from a packed byte buffer.** "size_t numberOfValues" is the count of integers to read. "uint8_t *bytes" points to the raw byte source. "int32_t *outputArray" is a caller-supplied array to fill with the results.
`for (size_t i = 0; i < numberOfValues; i++) {`	**Loop over each integer element.** size_t is used for loop counters that count elements or bytes, as it is guaranteed to be large enough to hold any array index on the current platform.
`size_t byteIndex = i * sizeof(int32_t);`	**Calculate the byte offset for element i.** Each 32-bit integer takes up exactly 4 bytes. The first integer starts at byte 0, the second at byte 4, the third at byte 8, and so on. Multiplying i by sizeof(int32_t) gives the correct starting byte address.
`int32_t value = readBytesAsInt32(&bytes[byteIndex]);`	**Delegate to the single-value helper.** &bytes[byteIndex] is the address of the byte at position byteIndex in the array — this is where the i-th integer starts in the buffer.

`outputArray[i] = value;`	**Store the result.** The decoded integer is placed into the caller's output array at position i.
`void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {`	**Write one 32-bit integer into a byte buffer.** The reverse of readBytesAsInt32. Takes an integer value and copies its 4 bytes into the byte buffer.
`memcpy(bytes, &value, sizeof(int32_t));`	**Copy the integer's bytes into the buffer.** &value is the address of the integer. memcpy copies sizeof(int32_t) bytes from that address into the buffer. The byte order (endianness) matches the host processor because both the write and the read use the same machine, so they are always consistent with each other.

> *The memcpy idiom avoids C's strict aliasing rule. Casting a float pointer to an int pointer and then reading through it (*(int*)&floatVar;) is technically undefined behaviour in C because the compiler is allowed to assume that an int pointer and a float pointer never point to the same memory. memcpy has no such restriction — it always does what the programmer intends, portably, on every compiler.*

4 · DataStorage.h

`src/tensor/include/DataStorage.h (struct definitions only — no .c file)`

DataStorage defines the two data structures (called structs in C) that back the **static arena allocator**. An arena allocator is a memory management strategy where a large block of memory is reserved once at program start-up, and individual allocations are made by simply advancing a pointer through that block — like dealing cards from the top of a deck. This is much faster than the general-purpose malloc() heap allocator, and it produces no memory fragmentation. On the STM32 Nucleo-F746ZG Microcontroller Unit, which has only 320 kilobytes of Static Random-Access Memory and no virtual memory system, fragmentation would quickly exhaust the available RAM. This file is a **pure header** — it contains only type definitions, not function implementations.

`typedef struct Entry {`	**Begin the Entry struct definition.** A struct in C groups several related variables (called members or fields) together under one name. "typedef" lets us write "dataEntry_t" instead of the longer "struct Entry" every time we use this type.
`uint8_t* dataPTR;`	**A pointer to the start of one allocation inside the arena.** uint8_t* means "pointer to an unsigned 8-bit integer" — effectively a raw memory address. The arena stores all data in one flat byte buffer, and this pointer records where a particular allocation begins inside that buffer.
`size_t numberOfElements;`	**The number of logical elements in this allocation.** This is a count of elements, not a count of bytes — for example, an array of 10 floats would have numberOfElements = 10. The actual byte size depends on the element type, which the caller knows.
`} dataEntry_t;`	**Close the struct and create the alias.** After this line, "dataEntry_t" is a valid type name for this struct.
`typedef struct DataStorage {`	**The arena allocator struct itself.** This struct represents the entire memory arena.
`uint8_t *data;`	**Pointer to the start of the backing byte array.** This is the large block of memory reserved at program start-up. All individual allocations live inside this buffer.
`size_t size;`	**Total capacity of the arena in bytes.** The arena cannot grow beyond this size. On the Microcontroller Unit, this is set to fit within the available Static Random-Access Memory.
`dataEntry_t *entries;`	**Array of entry records.** Each time a new allocation is made from the arena, a new dataEntry_t is added to this array, recording where the allocation starts and how many elements it contains.
`size_t numberOfEntries;`	**How many allocations are currently tracked.** This counter grows each time addDataToStorage is called.
`} dataStorage_t;`	**Close the struct and create the alias.**
`uint8_t* getDataFromStorage(dataStorage_t storage, void* dataPTR);`	**Lookup function declaration.** Given a raw pointer, find the corresponding entry in the arena's entry table. Returns the start of the allocation. "void*" is a generic pointer that can point to any type — explained further in the Syntax Quick Reference.
`dataEntry_t* addDataToStorage(dataStorage_t storage, void* dataPTR, size_t numberOfElements);`	**Registration function declaration.** Records a new allocation in the arena's entry table. Returns a pointer to the newly created dataEntry_t record.

The arena pattern eliminates heap fragmentation — critical on the STM32 Nucleo-F746ZG Microcontroller Unit which has only 320 kilobytes of Static Random-Access Memory and no virtual memory system. With a traditional heap, many small malloc/free cycles cause the available memory to become full of small unusable gaps, eventually causing allocation failures even when the total free memory is sufficient. The arena never has this problem because memory is only ever allocated, never individually freed.

5 · StorageApi.h / StorageApi.c

`src/userApi/include/StorageApi.h + src/userApi/StorageApi.c`

StorageApi is the **single entry point for all dynamic memory allocations** in the project. Instead of calling the C standard library's malloc() function directly, every module that needs memory calls reserveMemory() instead. On the Microcontroller Unit target, reserveMemory() internally maps to the static arena bump-allocator described in section 4. On a host Personal Computer build (used for testing and development), it maps to the standard malloc() and free() functions. This abstraction means the same application code runs on both platforms without any changes.

`void* reserveMemory(size_t size);`	**Allocate size bytes and return a pointer.** "void*" is a generic pointer type — the caller casts it to the specific type they need (e.g. float*, int32_t*). The function signature is deliberately identical to malloc() so that on host builds, the implementation can simply be "#define reserveMemory malloc". "size_t size" specifies how many bytes are requested.
`void freeReservedMemory(void* ptr);`	**Release a previously reserved block.** On a host Personal Computer build, this calls free(ptr) to return the memory to the heap. On the Microcontroller Unit arena build, this is a no-operation (a function that does nothing) — the arena never releases individual allocations; all memory is freed at once when the program ends or the arena is reset.

Every subsequent module allocates through reserveMemory(). Never call malloc() directly — it would bypass the arena on Microcontroller Unit targets and could cause fragmentation or allocation failures during long training runs.

6 · Quantization.h / Quantization.c

`src/tensor/include/Quantization.h + src/tensor/Quantization.c`

Quantization.h defines all quantization types and their configuration structs. **Quantization** is the process of mapping a floating-point number to an integer. For example, a weight value of 0.75 might be stored as the integer 24576 with a scale factor of 0.75/24576 ≈ 0.0000305. To recover the original value, multiply: 24576 × 0.0000305 ≈ 0.75. Storing weights as integers rather than floats saves memory and makes arithmetic faster on hardware that has no floating-point unit. The five quantization type values are: INT32 (raw 32-bit integer, no scale), FLOAT32 (native 32-bit float, no quantization), SYM_INT32 (symmetric quantization with int32 storage), SYM (symmetric 8-bit integer), and ASYM (asymmetric 8-bit integer with a zero-point offset).

`typedef enum qtype {`	**Begin the quantization type enumeration.** This enum will have one named value for each of the five supported quantization formats.
`INT32,`	**Quantization type 0 — Raw 32-bit integer, no scaling.** Values are stored as plain integers with no associated scale factor. Used for intermediate accumulator tensors inside matrix multiplication, where the results of integer multiplications are summed before any scale conversion is applied.
`FLOAT32,`	**Quantization type 1 — Native 32-bit floating-point.** No quantization is applied. Values are stored exactly as IEEE 754 single-precision floats. Used during the first phase of training and anywhere full numerical precision is needed.
`SYM_INT32,`	**Quantization type 2 — Symmetric quantization with 32-bit integer storage.** The real value equals the stored integer multiplied by a scale factor: real_value = stored_int32 × scale. The range is symmetric around zero (–max to +max). This is the primary format used during Fully Quantized Training forward and backward passes.
`SYM,`	**Quantization type 3 — Symmetric 8-bit integer.** Same as SYM_INT32 but stores values in 8 bits instead of 32 bits. This uses less memory but has lower precision. SYM_INT32 is preferred in this project.
`ASYM`	**Quantization type 4 — Asymmetric 8-bit integer with zero-point.** The real value equals (stored_int8 + zero_point) × scale. This allows the quantized range to be shifted away from zero, which is useful for activations (outputs of activation functions like Rectified Linear Unit) that are always non-negative. Used during asymmetric inference.
`} qtype_t;`	**Close the enum and create the alias qtype_t.**
`typedef struct symInt32QConfig {`	**Configuration struct for SYM_INT32 quantization.** Every SYM_INT32 tensor carries one of these to describe how its stored integers relate to real values.
`float scale;`	**The scale factor.** real_value = stored_int32 × scale. Initially set to 1.0, it is recomputed at quantization time based on the largest absolute value in the tensor (so the biggest value maps to the largest representable integer).
`roundingMode_t roundingMode;`	**Which rounding strategy to use.** Either Half-To-Even (deterministic) or Stochastic Rounding Half-To-Even (randomized). The choice affects whether tiny gradients are preserved.

`uint8_t qMaxBits;`	**Number of significant bits in the quantized representation.** Default is 16. With 16 bits, the maximum integer value is 2^(16-1) – 1 = 32767. This controls the precision-versus-range trade-off of the quantization.
`} symInt32QConfig_t;`	**Close the struct and create the alias.**
`typedef struct asymQConfig {`	**Configuration struct for asymmetric 8-bit integer quantization.**
`float scale;`	**Scale factor.** Same role as in SYM_INT32 — maps integers to real values.
`int16_t zeroPoint;`	**Zero-point offset.** A 16-bit signed integer that shifts the quantized range. For example, if the real values are always in [0, 1], the zero-point encodes the real zero so the full 8-bit range [0, 255] maps to [0, 1] without wasting half the range on negative values.
`uint8_t qBits;`	**Number of bits per element.** Typically 8, giving a range of 0–255 (unsigned) or –128–127 (signed).
`roundingMode_t roundingMode;`	**Rounding mode used when converting floats to this format.**
`} asymQConfig_t;`	**Close the struct and create the alias.**
`typedef struct quantization {`	**The universal quantization descriptor.** Every tensor carries one of these to describe its numerical format. It contains both which format is used (the "type" discriminator) and a pointer to the format-specific configuration.
`qtype_t type;`	**Which of the five quantization types this tensor uses.** This field acts as a discriminator — it tells downstream code which member of qConfig to expect.
`void* qConfig;`	**Generic pointer to the type-specific configuration struct.** "void*" is a pointer that can point to any type. For FLOAT32 and INT32, this is NULL (nothing) because no configuration is needed. For SYM_INT32, it points to a symInt32QConfig_t. For ASYM, it points to an asymQConfig_t. The caller casts it to the correct type based on the "type" field.
`} quantization_t;`	**Close the struct and create the alias.**

Key initialiser functions from Quantization.c:

`void initFloat32Quantization(quantization_t *quantization) {`	**Initialise a FLOAT32 quantization descriptor.** The simplest case — just set the type and leave the config pointer as NULL.
`quantization->type = FLOAT32;`	**Set the type discriminator to FLOAT32.** The "->" operator is used to access a member of a struct through a pointer. "quantization->type" means "the type field of the struct that quantization points to".
`quantization->qConfig = NULL;`	**No configuration struct needed for native floats.** NULL is a special pointer value meaning "points to nothing".
`void initSymInt32QConfig(roundingMode_t roundingMode, symInt32QConfig_t *symInt32QConfig) {`	**Fill in a SYM_INT32 configuration struct with default values.** The caller provides the rounding mode; scale and bit-width are given sensible defaults.
`symInt32QConfig->roundingMode = roundingMode;`	**Store the caller-chosen rounding mode.** This is either Half-To-Even or Stochastic Rounding Half-To-Even.
`symInt32QConfig->scale = 1.f;`	**Set initial scale to 1.0.** This is a placeholder. The real scale will be computed at quantization time based on the tensor's actual maximum value. The "f" suffix on 1.f tells the compiler this is a single-precision float literal (not a double).

`symInt32QConfig->qMaxBits = 16;`	**Default to 16 significant bits.** This gives a maximum representable integer of 32767, which provides good precision for gradient accumulation.
`void initAsymQConfig(uint8_t qBits, roundingMode_t roundingMode, asymQConfig_t *asymQConfig) {`	**Fill in an asymmetric configuration struct.**
`asymQConfig->qBits = qBits;`	**Store the requested bit-width** — typically 8.
`asymQConfig->scale = 1.f;`	**Placeholder scale.** Recomputed during conversion from float tensor to asymmetric tensor.
`asymQConfig->zeroPoint = (uint16_t)0;`	**Placeholder zero-point.** The cast (uint16_t) converts 0 to the correct integer type. This value is recomputed from the tensor's minimum value during conversion.

7 · Tensor.h / Tensor.c

`src/tensor/include/Tensor.h + src/tensor/Tensor.c`

tensor_t is **the central data structure of the entire project**. Every layer, every loss function, and every optimizer function works exclusively with tensor_t pointers. A tensor is a mathematical object that generalises vectors (one-dimensional arrays) and matrices (two-dimensional arrays) to any number of dimensions. In this codebase, a tensor is represented as a **flat byte buffer** (a contiguous block of raw memory) annotated with a shape descriptor that says how to interpret the flat memory as a multi-dimensional array, plus a quantization descriptor that says how to interpret the raw bytes as numbers.

Struct definitions — Tensor.h:

`typedef struct shape {`	**The shape descriptor struct.** Describes how many dimensions a tensor has and how large each dimension is. A 2D matrix with 3 rows and 4 columns would have numberOfDimensions=2 and dimensions=[3, 4].
`size_t numberOfDimensions;`	**The number of dimensions (called the rank).** A scalar has rank 0, a vector has rank 1, a matrix has rank 2, and so on. size_t is the standard unsigned integer type for sizes.
`size_t* dimensions;`	**Array of dimension sizes.** A pointer to an array of length numberOfDimensions. Each entry gives the size of that dimension. For a [784, 256] weight matrix, this array would contain [784, 256].
`size_t* orderOfDimensions;`	**Permutation array for transposition.** Normally orderOfDimensions[i] = i (identity permutation — dimension i maps to itself). When transposeTensor() is called, two entries are swapped, recording a logical transposition without moving any data in memory. For example, swapping entries 0 and 1 means "when you access row i, column j, actually look up row j, column i in the raw buffer". This makes transposition an O(1) operation (constant time regardless of tensor size).
`} shape_t;`	**Close the struct and create the alias.**
`typedef struct tensor {`	**The main tensor struct.** Groups the raw data buffer, the shape, the quantization type, and the sparsity mask together.
`uint8_t* data;`	**Raw backing storage.** uint8_t* is used because the element size varies: 4 bytes for float and int32, 1 byte for int8. Using a byte pointer (uint8_t*) lets the code handle all element sizes uniformly. The DTypes helper functions translate between raw bytes and typed values.
`shape_t* shape;`	**Pointer to the shape descriptor.** Shared between tensors that are views (aliases) of the same underlying data.

`quantization_t* quantization;`	**Quantization descriptor.** Tells every piece of code that reads this tensor how to interpret the raw bytes as numbers. Without this, you cannot know whether the bytes represent floats, integers, or scaled integers.
`sparsity_t* sparsity;`	**Future hook for Rigorous Lottery (RigL) sparse masks.** Currently set to NULL in all live tensors. Will be used in a future phase to track which weights are "active" (non-zero) and which are pruned to zero as part of sparse training.
`} tensor_t;`	**Close the struct and create the alias tensor_t.**
`typedef struct parameter {`	**Pairs a weight tensor with its gradient tensor.** Every trainable parameter in the network (weights and biases) is represented as a parameter_t.
`tensor_t* param;`	**The weight values.** These are updated by the optimizer after each training step.
`tensor_t* grad;`	**The accumulated gradient tensor.** During the backward pass, gradient values are added into this tensor. After the optimizer step, sgdZeroGrad() resets it to zero so it is ready for the next batch.
`} parameter_t;`	**Close the struct and create the alias parameter_t.**

Key utility functions — Tensor.c:

`size_t calcNumberOfElementsByShape(shape_t *shape) {`	**Calculate the total number of elements in a tensor.** For a matrix of shape [3, 4], this returns 12. For a vector of shape [784], this returns 784.
`size_t numElem = 1;`	**Start with 1 — the multiplicative identity.** We will multiply by each dimension size in turn.
`for (size_t i = 0; i < shape->numberOfDimensions; i++) {`	**Loop over every dimension.**
`numElem *= shape->dimensions[i];`	**Multiply by the size of dimension i.** After the loop, numElem equals the product of all dimension sizes, which is the total element count.
`return numElem;`	**Return the total count.**
`size_t calcBytesPerElement(quantization_t *quantization) {`	**Returns how many bytes each element occupies.** This depends on the quantization type — float and int32 take 4 bytes each, while 8-bit integers take 1 byte.
`switch (quantization->type) {`	**Switch on the quantization type.**
`case INT32: return sizeof(int32_t);`	**Raw 32-bit integer: 4 bytes.**
`case FLOAT32: return sizeof(float);`	**32-bit float: 4 bytes.**
`case SYM_INT32: return sizeof(int32_t);`	**Symmetric 32-bit integer: 4 bytes.**
`case ASYM: asymQConfig_t *aq = quantization->qConfig; return ceil((float)aq->qBits / 8.0f);`	**Asymmetric variable-width: round up to whole bytes.** 8 bits = 1 byte. 4 bits (half-byte) = 1 byte (rounded up). 12 bits = 2 bytes. The cast (float) converts qBits to a float so the division is floating-point rather than integer division.
`default: exit(1);`	**Unknown quantization type: abort the program.** This should never happen if the code is used correctly. exit(1) terminates the program with an error code.

`void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {`	**Logically transpose a tensor by swapping two dimension mappings.** This does NOT move any data. It only swaps two entries in the orderOfDimensions array, recording that logical dimension dim0Index now maps to the physical location of dim1Index and vice versa.
`size_t temp = tensor->shape->orderOfDimensions[dim0Index];`	**Save the current mapping for dim0.** Classic three-step swap using a temporary variable.
`tensor->shape->orderOfDimensions[dim0Index] = tensor->shape->orderOfDimensions[dim1Index];`	**Copy dim1's mapping into dim0's slot.**
`tensor->shape->orderOfDimensions[dim1Index] = temp;`	**Copy the saved dim0 mapping into dim1's slot.** The swap is now complete. This O(1) operation (constant time, regardless of how large the tensor is) enables matrix multiplication to treat a weight matrix as its own transpose during the forward pass and restore it instantly for the backward pass, without ever copying the data.
`void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape, quantization_t *quantization, sparsity_t *sparsity) {`	**Populate all four fields of a tensor_t at once.** All parameters are pointers — no copying of the underlying arrays occurs. This is just pointer assignment.
`tensor->data = data;`	**Point the tensor at its backing storage.**
`tensor->shape = shape;`	**Point the tensor at its shape descriptor.**
`tensor->quantization = quantization;`	**Point the tensor at its quantization descriptor.**
`tensor->sparsity = sparsity;`	**Point the tensor at its sparsity mask** — NULL if none.

> *Stack-allocated tensors are common in this codebase. A typical usage is: declare a raw float array on the stack (float buf[n];), declare a tensor struct (tensor_t t;), then call setTensorValues(&t;, (uint8_t*)buf, ...) to connect them. No heap allocation is involved. This is safe as long as the tensor is not used after the function that declared the stack array returns.*

8 · TensorConversion.h / TensorConversion.c

`src/tensor/include/TensorConversion.h + src/tensor/TensorConversion.c`

TensorConversion implements a **5×5 dispatch table** that maps every pair of quantization types to a conversion function. There are 5 quantization types, so there are 25 possible (input type, output type) combinations. Rather than writing a large if-else chain, the code stores 25 function pointers in a 2-dimensional array indexed by [input_type][output_type]. The public Application Programming Interface is a single call: convertTensor(input, output). This is the boundary layer between floating-point and integer arithmetic throughout the entire project.

`typedef void (*conversionFunction_t)(tensor_t* inputTensor, tensor_t* outputTensor);`	**Define a function-pointer type alias.** In C, a function pointer is a variable that stores the address of a function. "typedef void (*conversionFunction_t)(...)" creates a new type name "conversionFunction_t" for the type "pointer to a function that takes two tensor_t pointers and returns nothing (void)". Any conversion function that matches this signature can be stored in a variable of this type.
`void convertTensor(tensor_t* inputTensor, tensor_t* outputTensor);`	**Public Application Programming Interface.** The only function external code needs to call. It reads the quantization type of the input and output tensors, looks up the correct conversion function in the dispatch table, and calls it.
`extern conversionFunction_t conversionMatrix[5][5];`	**Declaration of the dispatch table.** "extern" means the actual array is defined in TensorConversion.c — this is just the declaration that tells other files the array exists. It is a 5×5 grid of function pointers, indexed by [input_quantization_type][output_quantization_type].

The dispatch table — from TensorConversion.c:

`conversionFunction_t conversionMatrix[5][5] = {`	**Define and initialise the 5×5 conversion table.** In C99, arrays of function pointers are initialised the same way as any other array.
`[INT32] = {`	**Row for tensors whose input type is INT32.** C99 "designated initialisers" allow specifying which array element to initialise by name rather than by position. [INT32] = 0, [FLOAT32] = 1, etc.
`[FLOAT32] = convertInt32TensorToFloatTensor,`	**INT32 → FLOAT32: cast each stored 32-bit integer to a float.** No scale factor is needed because INT32 has no scale — the integer value IS the real value.
`[SYM_INT32] = convertInt32TensorToSymInt32Tensor,`	**INT32 → Symmetric INT32: copy the data and set scale to 1.**
`[ASYM] = convertInt32TensorToAsymTensor,`	**INT32 → Asymmetric 8-bit integer: compute the value range, derive scale and zero-point, pack each integer into qBits.**
`[SYM] = unsupportedConversionTypes,`	**Not implemented — calling this prints an error and exits.**
`[INT32] = NULL,`	**Same-type diagonal — NULL because it is handled separately** by a function called convertTensorsWithSameType, which just copies the data without any format transformation.
`[FLOAT32] = {`	**Row for tensors whose input type is FLOAT32.**
`[SYM_INT32] = convertFloatTensorToSymInt32Tensor,`	**The most important conversion in Fully Quantized Training.** Converts each 32-bit float to a symmetric 32-bit integer using a per-tensor scale factor. Used to quantize activations and gradients before integer matrix multiplication.
`[ASYM] = convertFloatTensorToAsymTensor,`	**Float → asymmetric 8-bit integer** with both a scale and a zero-point offset.

The key float → Symmetric INT32 conversion function:

`void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	**Convert a FLOAT32 tensor to SYM_INT32 format.** This is the quantization step that turns floating-point values into integers.
`float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);`	**Find the largest absolute value in the tensor.** This defines the representable range — all values must fit between –absMax and +absMax.
`const float qMax = powf(2, (float)qMaxBits - 1) - 1;`	**Calculate the maximum representable quantized integer.** For qMaxBits = 16 bits: 2^(16–1) – 1 = 32767. For qMaxBits = 8 bits: 2^(8–1) – 1 = 127. powf() is the single-precision power function from math.h.
`scale = absMax / qMax;`	**Compute the scale factor.** Dividing absMax by qMax gives the scale such that absMax / scale = qMax exactly. Any input value divided by this scale and rounded will land within the range [–qMax, +qMax] and never overflow.
`outputSymInt32QC->scale = scale;`	**Store the computed scale in the output tensor's configuration** so that downstream code that reads the output tensor knows how to convert its integers back to real values (by multiplying by scale).
`outputInt32[i] = roundByMode(clamp(inputFloat[i] / scale, qMin, qMax), outputSymInt32QC->roundingMode);`	**For each element: divide by scale, clamp, then round.** Dividing by scale maps the float to the integer range. clamping ensures the value does not exceed the integer bounds even due to floating-point rounding errors. roundByMode applies either Half-To-Even or Stochastic Rounding Half-To-Even to convert the float to an integer.

9 · Arithmetic.h / Arithmetic.c

`src/arithmetic/include/Arithmetic.h + src/arithmetic/Arithmetic.c`

Arithmetic.c is the **dispatch engine for all element-wise operations** on tensors. An element-wise operation applies the same mathematical function independently to each pair of corresponding elements from two tensors — for example, adding tensor A to tensor B produces a result where result[i] = A[i] + B[i] for every index i. The file defines two function-pointer type aliases (one for 32-bit integer operations, one for float operations) and generic driver functions that iterate over tensor elements and apply whatever operator function was passed in. All Add, Subtract, Multiply, and Divide operations in the project are thin wrappers over these drivers. The driver correctly handles **transposition** by consulting the orderOfDimensions array when computing the physical memory address for each logical element.

`typedef int32_t(*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);`	**Define a function-pointer type for 32-bit integer element operations.** This is the type "pointer to a function that takes two 32-bit signed integers and returns one 32-bit signed integer". Any function with that signature — such as addInt32s(a, b) which returns a+b — can be stored in a variable of this type and passed to the generic driver.
`typedef float(*floatElementArithmeticFunc_t)(float a, float b);`	**Same pattern for 32-bit floating-point element operations.** A function pointer type for any function that takes two floats and returns one float.

Index helper — calcIndicesByRawIndex():

`void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t *indices) {`	**Convert a flat (linear) array index into a per-dimension index array.** In memory, a multi-dimensional array is stored as a flat sequence of elements. For a 2D matrix with 2 rows and 3 columns, element at row 1, column 1 is at flat index 1×3 + 1 = 4. This function does the reverse: given flat index 4 and dimensions [2,3], it produces indices [1, 1].
`size_t offset = 1;`	**Will hold the product of all dimensions (total element count).** Initialised to 1 (the multiplicative identity).
`for (size_t i = 0; i < numberOfDims; i++) { offset *= dims[i]; }`	**Compute the total number of elements.** For dims=[2,3], offset = 6.
`size_t restIndex = rawIndex;`	**Working copy of the flat index.** We peel off each dimension's contribution one at a time, reducing restIndex toward 0.
`for (size_t i = 0; i < numberOfDims; i++) {`	**Loop from the most-significant dimension to the least.**
`offset /= dims[i];`	**offset becomes the stride for dimension i.** The stride of a dimension is how many elements you skip in the flat array when you increment that dimension's index by 1. For dims=[2,3], at i=0: offset = 6/2 = 3 (each row contains 3 elements). At i=1: offset = 3/3 = 1.
`indices[i] = restIndex / offset;`	**Integer division gives the index for dimension i.** For rawIndex=4, dims=[2,3]: at i=0, indices[0] = 4/3 = 1. At i=1, indices[1] = 1/1 = 1. Result: [1, 1]. Correct!
`restIndex -= indices[i] * offset;`	**Remove this dimension's contribution from the remaining index.** restIndex = 4 – 1×3 = 1 after i=0. restIndex = 1 – 1×1 = 0 after i=1.

The generic float element-wise driver — floatPointWiseArithmetic():

`void floatPointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor, floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor) {`	**Apply an arbitrary element-wise function to two float tensors.** "arithmeticFunc" is a function pointer — the caller passes in whatever operation they want (add, subtract, multiply, divide). The driver iterates over all elements and calls the function once per pair of corresponding elements.
`if (!doDimensionsMatch(aTensor, bTensor)) { exit(1); }`	**Safety check — both tensors must have the same shape.** You cannot add a [3, 4] matrix to a [2, 5] matrix element-wise because the indices do not correspond. doDimensionsMatch() checks that all dimension sizes match.
`for (size_t i = 0; i < numberOfElements; i++) {`	**Iterate over all elements using a flat index.**
`calcIndicesByRawIndex(..., i, aIndices);`	**Convert flat index i to per-dimension indices for tensor A.**
`aElementIndex = calcElementIndexByIndices(..., aTensor->shape->orderOfDimensions);`	**Map logical indices back to a physical storage index,** respecting any transposition recorded in orderOfDimensions. If tensor A has been transposed, this function automatically swaps the indices before computing the flat address.
`float aValue = readBytesAsFloat(&aTensor->data[aByteIndex]);`	**Read element from tensor A's raw byte buffer.** aByteIndex = aElementIndex × sizeof(float). The DTypes helper converts the 4 raw bytes at that address back to a float.
`float bValue = readBytesAsFloat(&bTensor->data[bByteIndex]);`	**Read the corresponding element from tensor B.**
`float result = arithmeticFunc(aValue, bValue);`	**Apply the operation.** For example, if arithmeticFunc is addFloat32s, this computes aValue + bValue.
`writeFloatToByteArray(result, &outputTensor->data[outputByteIndex]);`	**Write the result back to the output tensor's byte buffer.**

> *The InPlace variants (e.g. addFloat32TensorsInplace) write results back into aTensor->data instead of a separate outputTensor, saving a memory allocation. They are used in the Linear layer bias addition (output += bias) to avoid allocating a temporary tensor.*

10 · Matmul.h / Matmul.c

`src/arithmetic/include/Matmul.h + src/arithmetic/Matmul.c`

Matmul implements **2-dimensional matrix multiplication** for float32, int32, and symmetric int32 tensors. Matrix multiplication (matmul) is the most computationally expensive operation in the training loop — every forward pass and every backward pass through a Linear layer calls it. Given a matrix A of shape [m, k] and a matrix B of shape [k, n], the output has shape [m, n] where output[row][col] = sum of A[row][i] × B[i][col] for i = 0 to k–1 (a dot product of a row of A with a column of B). The symmetric int32 variant performs integer multiplication and then propagates the scale factor: if A has scale_A and B has scale_B, the output's scale is scale_A × scale_B.

`void matmulFloatTensors(tensor_t *aTensor, tensor_t *bTensor, tensor_t *outputTensor) {`	**Core float matrix multiplication: output = A × B.**
`if (aColumns != bRows) { exit(1); }`	**Dimension compatibility check.** Matrix multiplication requires that the number of columns in A equals the number of rows in B. For example, [3×4] × [4×5] is valid (inner dimension = 4 on both sides), but [3×4] × [5×4] is not. If incompatible dimensions are detected, the program aborts immediately.
`for (size_t rowIndex = 0; rowIndex < aRows; rowIndex++) {`	**Outer loop over rows of A** — these become rows of the output.

`for (size_t columnIndex = 0; columnIndex < bColumns; columnIndex++) {`	**Middle loop over columns of B** — these become columns of the output.
`float result = 0;`	**Accumulator for the dot product.** Start at zero and add each product term.
`for (size_t i = 0; i < aColumns; i++) {`	**Inner loop — dot product over the shared dimension.** This is the innermost loop and runs the most frequently. For large matrices, this loop dominates execution time.
`size_t aIndices[] = {rowIndex, i};`	**Logical index into A: row rowIndex, column i.**
`size_t aValueIndex = calcElementIndexByIndices(..., aTensor->shape->orderOfDimensions);`	**Compute the physical storage index for A[rowIndex][i].** The orderOfDimensions array is consulted here, so if the matrix has been transposed (by calling transposeTensor), the indices are automatically swapped before the physical address is computed.
`float aValue = readBytesAsFloat(&aTensor->data[aByteIndex]);`	**Read element A[rowIndex][i] from raw memory.**
`result += mulFloat32s(aValue, bValue);`	**Multiply and accumulate.** mulFloat32s is a thin wrapper around the float multiplication operator. Adding to result builds up the dot product.
`writeFloatToByteArray(result, &outputTensor->data[outputByteIndex]);`	**Write output[rowIndex][columnIndex] to the output tensor.** This completes one element of the result matrix.
`void matmulSymIntTensors(tensor_t *aTensor, tensor_t *bTensor, tensor_t *outputTensor) {`	**Symmetric INT32 matrix multiplication.** Reuses the integer matmul then propagates the scale factors.
`matmulInt32Tensors(aTensor, bTensor, outputTensor);`	**Perform integer matrix multiplication.** Same triple-nested loop but with 32-bit integer elements instead of floats.
`outputSymInt32QC->scale = aSymInt32QC->scale * bSymInt32QC->scale;`	**Propagate the scale factor.** If A stores values with scale_A (meaning real_A = int_A × scale_A) and B stores values with scale_B, then the integer product int_A × int_B represents real_A × real_B = int_A × scale_A × int_B × scale_B = (int_A × int_B) × (scale_A × scale_B). So the output scale must be scale_A × scale_B.

The macro MATMUL_FUNC_FLOAT at the top of the file lets you substitute a different implementation by compiling with -DTRACK_INSTRUCTIONS. This swaps in an instruction-counting variant that records how many arithmetic operations were executed — useful for profiling how efficiently the Microcontroller Unit performs the matrix multiplication.

11 · RNG.h / RNG.c

`src/rng/include/RNG.h + src/rng/RNG.c`

The Random Number Generator is a lightweight **Exclusive-OR shift Pseudo-Random Number Generator** (often called an XOR-shift generator). A Pseudo-Random Number Generator is an algorithm that produces a sequence of numbers that appear random but are entirely determined by an initial "seed" value. The XOR-shift algorithm is very fast — it requires only three bitwise operations and no division or multiplication — making it ideal for the Microcontroller Unit where clock cycles are precious. It is used by Stochastic Rounding Half-To-Even (for the random dither) and by the dataset loader (for shuffling training samples). The generator is NOT thread-safe — it stores its state in a single global variable, which would cause data races if called simultaneously from multiple threads.

`typedef struct { uint32_t state; } rng32_t;`	**The entire generator state is a single 32-bit unsigned integer.** All the randomness of the generator comes from transforming this one number. uint32_t is an unsigned 32-bit integer — it holds values from 0 to 4,294,967,295 (2^32 – 1). The typedef wraps it in a struct so that it can be passed by pointer and its type is clear.
`static rng32_t rng = {.state = 1};`	**The module-global generator instance.** "static" at file scope means this variable exists for the lifetime of the program but is invisible outside RNG.c (it is "file-private"). The initializer {.state = 1} is C99 designated initializer syntax that sets the state field to 1. The state must never be 0 — an XOR-shift generator with state 0 would produce 0 forever because 0 XOR anything = 0.
`static uint32_t rngNext(rng32_t *rng) {`	**Core XOR-shift generator — file-private.** "static" on a function means it is invisible outside this .c file. It takes a pointer to the generator state and returns the next pseudo-random 32-bit integer.
`uint32_t x = rng->state;`	**Copy the current state into a local variable.**
`x ^= x << 13;`	**Left-shift by 13 bits, then Exclusive-OR with original.** Exclusive-OR (XOR, written ^) compares bits: output bit is 1 if the two input bits differ, 0 if they are the same. Shifting left by 13 moves all bits 13 positions to the left (bits shifted off the left edge are lost; zeros fill from the right). XOR-ing the shifted value with the original spreads information from low-order bits upward.
`x ^= x >> 17;`	**Right-shift by 17 bits, then Exclusive-OR.** Shifting right by 17 moves bits downward. XOR-ing spreads information from high-order bits downward.
`x ^= x << 5;`	**Left-shift by 5 bits, then Exclusive-OR — final mixing step.** Together the three XOR-shift operations form the xorshift32 algorithm published by George Marsaglia in 2003. The constants 13, 17, and 5 were chosen by Marsaglia to give the generator its maximum possible period of 2^32 – 1 (it cycles through all 4,294,967,295 non-zero 32-bit values before repeating).
`rng->state = x;`	**Update the state with the new value.** This mutates the generator's state so the next call produces a different result.
`return x;`	**Return the new pseudo-random 32-bit unsigned integer.**
`float rngNextFloat(void) {`	**Convert a 32-bit integer to a float in the range [0, 1).** "void" as the parameter list means the function takes no arguments.

`return (float)(rngNext(&rng) >> 8) / (float)(1 << 24);`	**Discard 8 low-order bits, then divide by 2^24.** Right-shifting by 8 produces a 24-bit number (values 0 to 2^24 – 1). Dividing by 2^24 maps it to the range [0, 1). Using 24 bits matches the 24-bit mantissa of a 32-bit float, so the result uses the full precision of the float type. This float is used as the dither in Stochastic Rounding.
`void rngShuffleIndices(size_t *indices, size_t n) {`	**Fisher-Yates shuffle of an index array.** After this function, the elements of the indices array are in a uniformly random order. The Fisher-Yates algorithm is the standard unbiased shuffle — every permutation is equally likely.
`for (size_t i = n - 1; i > 0; --i) {`	**Iterate from the last element down to index 1.** The "--i" (pre-decrement) subtracts 1 from i before using it in the loop body.
`size_t j = rngBounded(&rng, i + 1);`	**Pick a random index j in [0, i] — inclusive on both ends.** rngBounded() uses rejection sampling (described in the note below) to ensure j is perfectly uniformly distributed in [0, i], with no bias toward small values.
`size_t tmp = indices[i]; indices[i] = indices[j]; indices[j] = tmp;`	**Swap elements at positions i and j.** The classic three-variable swap: save i's value, copy j's value into i's slot, copy the saved value into j's slot.

rngBounded() uses rejection sampling to eliminate modulo bias. If you compute (randomInt % n) to get a value in [0, n), values near 0 are slightly more likely than values near n–1 when 2^32 is not exactly divisible by n. Rejection sampling loops (do{}while x >= limit) until it gets a value that falls in an unbiased range, then uses it. This ensures perfectly fair dataset shuffling even when the number of training samples is not a power of 2.

Phase 6 — Layer System

Layer · Linear · ReLU · Softmax · QuantizationLayer

12 · Layer.h / Layer.c

`src/layer/include/Layer.h + src/layer/Layer.c`

Layer.h implements **C-style polymorphism** — a way to write code that works with different types of layers (Linear, Rectified Linear Unit, Softmax, etc.) without knowing at compile time which layer type will be used. In object-oriented languages like C++ or Python, this is done with virtual methods on base classes. In C, it is done with a "vtable" — an array of function pointers where each row contains the forward, backward, and shape-calculation functions for one layer type. A layer_t struct stores a type enum (an integer that says which row of the vtable to use) and a config pointer. The training loop calls layerFunctions[layer->type].forward(layer, input, output) and the correct function runs, regardless of which layer type is active.

`typedef enum layerType {`	**Enumerate all supported layer types.** Each named value is an integer that serves as an index into the layerFunctions[] vtable array.
`LINEAR, RELU, CONV1D, SOFTMAX, SEQUENTIAL, QUANTIZATION`	**Six layer type values.** LINEAR = 0 (fully-connected layer), RELU = 1 (Rectified Linear Unit activation), CONV1D = 2 (one-dimensional convolution, currently a stub), SOFTMAX = 3, SEQUENTIAL = 4 (a container that holds a sequence of other layers), QUANTIZATION = 5 (the Fully Quantized Training insertion point).
`} layerType_t;`	**Close the enum and create the alias.**
`typedef union layerConfig {`	**A union that holds one of several configuration pointers.** A C union is like a struct except all its members share the same memory address — only one member is valid at any given time. This saves memory: instead of storing a pointer for every possible layer type, only one pointer's worth of memory is used.
`linearConfig_t* linear;`	**Valid when type == LINEAR.**
`reluConfig_t* relu;`	**Valid when type == RELU.**
`softmaxConfig_t* softmax;`	**Valid when type == SOFTMAX.**
`} layerConfig_t;`	**Unions save memory** because a pointer is the same size (4 or 8 bytes depending on the platform) regardless of which config type it points to.
`typedef struct layer {`	**The main layer struct.**
`layerType_t type;`	**The type discriminator.** This integer tells the training loop which row of layerFunctions[] to use, and which member of the config union is currently valid.
`layerConfig_t* config;`	**Pointer to the type-specific configuration** (allocated via reserveMemory()).
`} layer_t;`	**Close the struct and create the alias.**
`typedef void (*forwardFn_t)(layer_t*, tensor_t*, tensor_t*);`	**Function-pointer type for forward pass functions.** Every forward function takes the layer, its input tensor, and its output tensor.
`typedef void (*backwardFn_t)(layer_t*, tensor_t*, tensor_t*, tensor_t*);`	**Function-pointer type for backward pass functions.** Takes the layer, the forward-pass input (needed to compute weight gradients), the upstream loss gradient (from the next layer), and writes the propagated loss (sent to the previous layer).

`typedef void (*calcOutputShapeFn_t)(layer_t*, shape_t*, shape_t*);`	**Function-pointer type for output shape calculation.** Given the layer and the input shape, computes the output shape. Used during network setup to pre-allocate all output tensors before training begins.
`typedef struct layerFunctions {`	**One row of the vtable — stores the three function pointers for one layer type.**
`forwardFn_t forward; backwardFn_t backward; calcOutputShapeFn_t calcOutputShape;`	**The three function pointers.**
`} layerFunctions_t;`	**Close the struct and create the alias.**
`extern layerFunctions_t layerFunctions[];`	**Declare the global vtable array.** "extern" means the actual array is defined in Layer.c. Any file that includes Layer.h can use this declaration to call into the vtable without knowing the concrete layer type.

Layer.c — the vtable definition:

`layerFunctions_t layerFunctions[] = {`	**Define the global vtable, initialised with designated initialisers.**
`[LINEAR] = {linearForward, linearBackward, linearCalcOutputShape},`	**Wire the LINEAR vtable row to the concrete functions in Linear.c.** When the training loop calls layerFunctions[LINEAR].forward(...), it calls linearForward(...).
`[RELU] = {reluForward, reluBackward, reluCalcOutputShape},`	**Wire the RELU row to Relu.c functions.**
`[CONV1D] = {NULL, NULL, NULL},`	**Convolution is a stub — all three pointers are NULL.** Calling any of these would dereference a NULL pointer and crash the program. Guard code should check for NULL before calling.
`[SOFTMAX] = {softmaxForward, softmaxBackward, softmaxCalcOutputShape},`	**Wire the SOFTMAX row to Softmax.c functions.**
`void initLayer(layer_t *layer, layerType_t type, layerConfig_t *config) {`	**Trivial layer constructor — sets the two fields.**
`layer->type = type; layer->config = config;`	**Store the type discriminator and the config pointer.**

13 · Linear.h / Linear.c

`src/layer/include/Linear.h + src/layer/Linear.c`

The Linear layer implements the **fully-connected (dense) layer**: output = Weights × input + bias. This is the fundamental building block of multi-layer perceptrons. The weight matrix has shape [output_size, input_size] and the bias vector has shape [output_size]. The forward pass transposes W (swapping row/column indices without copying data) then calls matmul. The backward pass computes three gradients: (1) dL/dW = the gradient of the loss with respect to the weights, (2) dL/db = the gradient with respect to the bias, and (3) dL/dx = the gradient propagated to the previous layer. Both float32 and Symmetric INT32 code paths are implemented, with "WithConversion" wrappers that handle the case where the input and output tensors are not already in the expected format.

`typedef struct linearConfig {`	**Configuration struct for a Linear layer.**
`parameter_t* weights;`	**Weight matrix W.** The param field holds the current weight values; the grad field accumulates dL/dW (the weight gradient) during the backward pass.
`parameter_t* bias;`	**Bias vector b.** param holds the bias values; grad accumulates dL/db (the bias gradient).
`quantization_t* forwardQ;`	**Quantization type for the forward pass output tensor.** If this is Symmetric INT32, the forward output is quantized to integers.

`quantization_t* weightGradQ;`	**Quantization type for the weight gradient tensor.** Can be different from the forward quantization — for example, you can run float forward but integer gradients.
`quantization_t* biasGradQ;`	**Quantization type for the bias gradient tensor.**
`quantization_t* propLossQ;`	**Quantization type for the propagated loss tensor** sent to the previous layer.
`} linearConfig_t;`	**Having four separate quantization types** lets you mix precision freely — for example, run float activations but integer gradients to reduce memory during backpropagation.

Forward pass — float code path:

`void linearForwardFloat(tensor_t *w, tensor_t *b, tensor_t *input, tensor_t *output) {`	**Compute output = W × input + bias using floating-point arithmetic.**
`transposeTensor(w, 0, 1);`	**Temporarily transpose W from shape [output_size, input_size] to [input_size, output_size].** Matrix multiplication requires that the number of columns in the left matrix equals the number of rows in the right matrix. The input has shape [1, input_size] (a row vector). To compute input × W^T and get [1, output_size], W must be presented as [input_size, output_size]. transposeTensor does this in constant time by swapping two entries in orderOfDimensions.
`matmulFloat32Tensors(input, w, output);`	**output = input × W^T.** After transposition, W has shape [input_size, output_size], so the multiplication input[1, input_size] × W[input_size, output_size] = output[1, output_size] gives one output row for each input row.
`transposeTensor(w, 0, 1);`	**Restore W to its original shape [output_size, input_size].** The transpose is non-destructive — only the orderOfDimensions entries are swapped back. The actual weight data is unchanged.
`addFloat32TensorsInplace(output, b);`	**Add the bias: output += b.** This is a broadcast addition — the same bias vector is added to every row of the output.

Backward pass — weight gradient (float code path):

`void linearCalcWeightGradsFloat32(tensor_t *forwardInput, tensor_t *loss, tensor_t *weightGrads) {`	**Compute dL/dW = loss^T × input.** By the chain rule, the gradient of the loss with respect to the weight matrix W equals the outer product of the upstream loss gradient vector and the forward input vector. For a batch, this is a matrix multiplication.
`transposeTensor(loss, 0, 1);`	**Transpose the loss from shape [1, output_size] to [output_size, 1].** This is needed so the matrix multiplication loss^T × input produces a result of shape [output_size, input_size], which matches the weight matrix shape.
`matmulFloat32Tensors(loss, forwardInput, &intermediateWGrad);`	**intermediateWGrad = loss^T × input = dL/dW for this sample.**
`transposeTensor(loss, 0, 1);`	**Restore the loss tensor's orientation** for subsequent use.
`addFloat32TensorsInplace(weightGrads, &intermediateWGrad);`	**Accumulate the gradient.** Gradients are summed over all samples in the batch. The optimizer step runs once at the end of the batch using the accumulated total.

Backward dispatch — linearBackward():

`void linearBackward(layer_t *linearLayer, tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {`	**Master backward function — dispatches to the right quantization path.**
`switch (linearConfig->weightGradQ->type) {`	**Check which quantization type the weight gradient uses** and call the corresponding implementation.
`case FLOAT32: linearCalcWeightGradsFloatWithConversion(linearConfig, forwardInput, loss); break;`	**The ...WithConversion suffix** means this function checks whether the input tensors are already in the expected format. If not, it converts them first (e.g. from Symmetric INT32 to float), computes the gradient in that format, then converts the result back.
`case SYM_INT32: linearCalcWeightGradsSymInt32WithConversion(linearConfig, loss, forwardInput); break;`	**Symmetric INT32 gradient path** — computes gradients in integer arithmetic for reduced memory usage during backpropagation.
`// identical switches for biasGradQ and propLossQ`	**The bias gradient and the propagated loss** are each computed with their own switch, independently of each other and of the weight gradient quantization.

14 · Relu.h / Relu.c

`src/layer/include/Relu.h + src/layer/Relu.c`

The Rectified Linear Unit activation is the simplest non-linear layer. It is **stateless** — it has no trainable parameters (no weights or biases). The forward pass applies max(0, x) element-wise: any negative value is replaced by zero, positive values pass through unchanged. The backward pass is the "straight-through" gradient: the upstream gradient passes through unchanged wherever the forward input was positive, and is set to zero wherever the forward input was zero or negative. This is the exact derivative of max(0, x), which is 1 for positive inputs and 0 for non-positive inputs.

`void reluForwardFloat(tensor_t *input, tensor_t *output) {`	**Float forward pass of Rectified Linear Unit.**
`gteFloatValue(input, 0, 0, output);`	**gteFloatValue (from Comparison.c) applies element-wise max(x, 0).** "gte" stands for "greater than or equal to". For each element: if input[i] >= 0, output[i] = input[i]; otherwise output[i] = 0. This is equivalent to max(x, 0) for every element.
`void reluBackwardFloat(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss) {`	**Float backward pass of Rectified Linear Unit.** Needs the original forward input to know which elements were positive.
`for (size_t i = 0; i < numberOfElements; i++) {`	**Iterate over every element.**
`if (inputArray[i] <= 0) {`	**If the forward input was zero or negative...** The Rectified Linear Unit output was 0 in this case. Its derivative is 0, so no gradient flows back through this element.
`gradInArray[i] = 0;`	**Zero out this element of the gradient.** The gradient is "blocked" — it does not propagate to earlier layers for this element.
`} else {`	**The forward input was positive...**
`gradInArray[i] = gradOutArray[i];`	**Pass the upstream gradient through unchanged.** The Rectified Linear Unit was linear (output = input) in this region, so its derivative is 1, and multiplying the upstream gradient by 1 gives the upstream gradient itself.

> *This is the Rectified Linear Unit "straight-through gradient" — the gradient is either passed straight through (where the forward input was positive) or blocked completely (where it was zero or negative). This is the exact mathematical derivative of the piecewise-linear Rectified Linear Unit function.*

15 · Softmax.h / Softmax.c

`src/layer/include/Softmax.h + src/layer/Softmax.c`

Softmax converts a vector of raw, unconstrained output values (called logits) into a valid probability distribution — a vector of non-negative values that sum to exactly 1.0. The formula is: softmax(x_i) = exp(x_i) / sum(exp(x_j) for all j). The implementation uses the **numerically stable log-sum-exp trick**: before computing the exponentials, the maximum value in the input is subtracted from every element. This does not change the output probabilities (the max cancels in the ratio) but prevents the exponentials from overflowing to infinity when the logits are large.

`static void softmaxForwardFloat(tensor_t *input, tensor_t *output) {`	**Float-path Softmax forward pass.** "static" = file-private.
`float *x = (float *)input->data;`	**Cast the raw byte pointer to a float pointer.** input->data is a uint8_t* (byte pointer). Casting to (float*) tells the compiler to interpret those bytes as 32-bit floats. This cast is valid here because the tensor's quantization type is FLOAT32, so the bytes really are IEEE 754 floats.
`float max = x[0]; for (size_t i = 1; i < n; i++) { if (x[i] > max) max = x[i]; }`	**Step 1: Find the maximum logit.** Scanning all elements to find the largest value. This is used in the numerically stable trick described above.
`float sum = 0.f; for (size_t i = 0; i < n; i++) { float e = expf(x[i] - max); y[i] = e; sum += e; }`	**Step 2: Compute exp(x_i – max) for each element and accumulate the sum.** Subtracting max ensures the argument to expf() is at most 0 (since x[i] - max <= 0 for all i), so expf() returns at most 1.0 and never overflows. expf() is the single-precision version of the exponential function from math.h.
`for (size_t i = 0; i < n; i++) { y[i] /= sum; }`	**Step 3: Divide each exponential by the total sum.** After this, y[i] = exp(x_i – max) / sum(exp(x_j – max)). The max cancels in the ratio (dividing numerator and denominator by exp(max) gives the original softmax formula), and all y[i] sum to 1.0.
`static void softmaxBackwardFloat(tensor_t *input, tensor_t *loss, tensor_t *propLoss) {`	**Jacobian-vector product for Softmax backward pass.**
`float dot = 0.0f; for (size_t i = 0; i < n; i++) dot += s[i] * dLds[i];`	**Compute the dot product of the Softmax output and the upstream gradient.** s[i] is the softmax output (the probability for class i). dLds[i] is the upstream gradient dL/ds_i. The dot product is a scalar used in the Jacobian formula.
`for (size_t i = 0; i < n; i++) dLdx[i] = s[i] * (dLds[i] - dot);`	**dL/dx_i = s_i × (dL/ds_i – dot).** This is the exact Jacobian-vector product of the Softmax function. In practice, when Softmax is immediately followed by Cross-Entropy loss, the fused backward function (see section 18) simplifies this to s – y_true, which is much cheaper to compute.

> *When Softmax is followed by Cross-Entropy loss, crossEntropySoftmaxBackward replaces this backward pass and computes the fused gradient (softmax_output – true_label) directly — more numerically stable and requiring only O(n) operations (linear time) instead of the full Jacobian matrix-vector product.*

16 · QuantizationLayer.c

`src/layer/QuantizationLayer.c`

QuantizationLayer is the **Fully Quantized Training insertion point**. In Fully Quantized Training, quantization operations are inserted into the computation graph as differentiable layers so that gradients can flow through them. The **forward pass** converts a tensor from one quantization type to another (for example, from float32 to Symmetric INT32) using the TensorConversion dispatch table. The **backward pass** uses the **straight-through estimator**: the upstream gradient is passed back through the quantization layer without any transformation, as if the quantization operation were an identity function (y = x). This approximation makes the quantization operation differentiable and allows end-to-end training with

integer arithmetic.

`void quantization(tensor_t *input, tensor_t *output) {`	**Forward pass — performs the quantization conversion.**
`qtype_t inputQType = input->quantization->type;`	**Read the input tensor's current quantization type.** This is an integer (0 through 4) indexing the quantization type enumeration.
`qtype_t outputQType = output->quantization->type;`	**Read the desired output quantization type** from the output tensor's quantization descriptor. The caller sets this up before calling the layer.
`conversionFunction_t conversion = conversionMatrix[inputQType][outputQType];`	**Look up the correct conversion function** in the 5×5 dispatch table from TensorConversion.c. For example, if inputQType = FLOAT32 and outputQType = SYM_INT32, this retrieves convertFloatTensorToSymInt32Tensor.
`conversion(input, output);`	**Call the conversion function.** This performs the actual quantization — mapping float values to scaled integers, computing the scale factor, and rounding using the configured rounding mode (Half-To-Even or Stochastic Rounding Half-To-Even).
`// Backward pass: copy upstream gradient directly to propLoss (straight-through estimator)`	**The backward pass is a no-operation (a pass-through).** Mathematically, quantization is not differentiable at the rounding points (the gradient is zero almost everywhere and undefined at integer boundaries). The straight-through estimator ignores this and pretends the gradient of "round(x)" is 1, meaning the gradient flows straight through the quantization layer unchanged. This approximation allows the weights below the quantization layer to receive useful gradient signals and learn.

Phase 7 — Loss Functions

LossFunction · CrossEntropy · Mean Squared Error

17 · LossFunction.h / LossFunction.c

`src/loss_functions/include/LossFunction.h + LossFunction.c`

LossFunction defines the **polymorphic loss interface** using the same dispatch-table pattern as the Layer system. A loss function measures how far the network's predictions are from the correct answers. The forward pass computes a scalar number (the loss value) that we want to minimise. The backward pass computes the gradient of the loss with respect to the model's output — this gradient is the starting point for backpropagation through all the layers.

`typedef float (*lossFwdFn_t)(tensor_t* modelOutput, tensor_t* label);`	**Function-pointer type for loss forward pass functions.** Takes the model's output tensor and the ground-truth label tensor. Returns a single float — the scalar loss value. This number is logged during training to track learning progress.
`typedef void (*lossBwdFn_t)(tensor_t* modelOutput, tensor_t* label, tensor_t* result);`	**Function-pointer type for loss backward pass functions.** Takes the model output, the label, and fills "result" with dL/d(modelOutput) — the gradient of the loss with respect to the model's output. This gradient flows into the last layer's backward pass.
`typedef struct lossFunctions {`	**One row of the loss vtable.**
`lossFwdFn_t forward; lossBwdFn_t backward;`	**The forward and backward function pointers for one loss type.**
`} lossFunctions_t;`	**Close the struct and create the alias.**
`typedef enum lossType { MSE, CROSS_ENTROPY } lossType_t;`	**Two supported loss types.** Mean Squared Error (value 0) is used for regression problems (predicting continuous values). Cross-Entropy (value 1) is used for classification problems (predicting one of several discrete categories).
`extern lossFunctions_t lossFunctions[];`	**The global loss vtable — indexed by lossType_t.**
`lossFunctions_t lossFunctions[] = {`	**Define and fill the vtable.**
`[MSE] = {mseLossForward, mseLossBackward},`	**Wire the Mean Squared Error row** to its implementation functions.
`[CROSS_ENTROPY] = {crossEntropyForwardFloat, crossEntropySoftmaxBackward},`	**Wire the Cross-Entropy row.** Note that the backward function is the fused Softmax + Cross-Entropy backward pass, not the standalone Cross-Entropy backward. This is more numerically stable and more efficient.
`// lossFunctions[lossType].forward(output, label)`	**The training loop calls the loss this way** — it does not need to know which loss type is active, because the vtable handles the dispatch automatically.

18 · CrossEntropy.c

`src/loss_functions/CrossEntropy.c`

Cross-entropy loss for classification. The **forward pass** computes –sum(y_true_i × log(y_hat_i)) where y_hat is the softmax output (predicted probabilities) and y_true is the ground-truth label (a one-hot vector, or a soft label). This quantity is zero when the model is perfectly confident and correct, and grows toward infinity as the model becomes less confident. The **backward pass** uses the mathematically simplified fused Softmax + Cross-Entropy gradient: dL/d(logits) = y_hat – y_true. This elegant result comes from applying the chain rule to the composition of the two operations.

`float crossEntropyForwardFloat(tensor_t *softmaxOutput, tensor_t *distribution) {`	**Compute the cross-entropy loss given probabilities and labels.**

`float *p = (float *)softmaxOutput->data;`	**Cast raw bytes to float pointer.** p[i] is the predicted probability for class i (after Softmax). Valid because the tensor is FLOAT32 type.
`float *y = (float *)distribution->data;`	**y[i] is the ground-truth label for class i.** For one-hot labels, only one y[i] is 1 and all others are 0. For soft labels, all y[i] are between 0 and 1 and sum to 1.
`float loss = 0.f;`	**Accumulator for the loss value, starting at 0.**
`for (size_t i = 0; i < n; i++) {`	**Loop over every class.**
`float pi = fmaxf(p[i], 1e-7f);`	**Clamp the predicted probability away from zero.** fmaxf returns the larger of its two arguments (float version). log(0) = negative infinity, which would produce Not-a-Number (an invalid IEEE 754 float) and corrupt the loss computation. The tiny constant 1e-7 (0.0000001) ensures pi is never exactly 0.
`loss += y[i] * -logf(pi);`	**Add this class's contribution to the loss.** logf() is the natural logarithm (base e) of pi. Since pi is in (0, 1], logf(pi) is in (–infinity, 0]. Negating it makes it non-negative. Multiplying by y[i] weights the contribution by how much probability the true label assigned to this class — for one-hot labels, only the true class contributes.
`return loss;`	**Return the scalar loss value.** This number is logged but not used in backpropagation — the backward function computes the gradient directly without needing this value.
`static void crossEntropySoftmaxBackwardFloat(...) {`	**Fused Softmax + Cross-Entropy backward pass.** "static" = file-private.
`for (size_t i = 0; i < totalInputSize; i++) {`	**Loop over every element.**
`lossFloat[i] = softmaxOutputFloat[i] - distributionFloat[i];`	**The gradient is (predicted probability – true label).** For one-hot labels, this is softmax_output[i] – 1 for the true class, and softmax_output[i] – 0 = softmax_output[i] for all other classes. This beautifully simple result arises from applying the chain rule to the Softmax and Cross-Entropy operations together — the Softmax Jacobian cancels with terms from the Cross-Entropy gradient, leaving just this subtraction.

19 · Sgd.h / Sgd.c

`src/optimizer/include/Sgd.h + src/optimizer/Sgd.c`

Stochastic Gradient Descent with optional momentum and weight decay. "Stochastic" means the gradient is estimated from a small random subset (a mini-batch) of the training data rather than the full dataset. Two update rules are implemented: (1) Vanilla Stochastic Gradient Descent: subtract a small multiple (the learning rate) of the gradient from each weight. (2) Stochastic Gradient Descent with Momentum: instead of following the gradient directly, maintain a running "velocity" vector that accumulates past gradients (like a ball rolling downhill gaining speed). Momentum smooths out the oscillations that can occur in Stochastic Gradient Descent and generally reaches a good solution faster.

`typedef struct sgd {`	**Configuration struct for the Stochastic Gradient Descent optimizer.**
`float learningRate;`	**The learning rate (step size).** A typical value is 0.01 or 0.001. It controls how far each weight moves in the gradient direction per update. Too large: weights overshoot and training diverges. Too small: training takes a very long time to converge.
`float momentumFactor;`	**The momentum coefficient (beta).** A typical value is 0.9. At 0.0, there is no momentum and the optimizer behaves like vanilla Stochastic Gradient Descent. At 0.9, the velocity vector retains 90% of its previous value and adds 10% of the new gradient — a running average with a long memory.
`float weightDecay;`	**Weight decay coefficient (L2 regularisation).** At each step, the gradient is augmented by weightDecay × weight. This gently pulls all weights toward zero, discouraging the network from relying on very large weights. It acts as a penalty on the magnitude of the weights (L2 regularisation).
`} sgd_t;`	**Close the struct and create the alias.**

Vanilla Stochastic Gradient Descent step (float code path):

`static void sgdStepFloat(optimizer_t *optim) {`	**Perform one parameter update step.** "static" = file-private.
`sgd_t *sgd = (sgd_t *)optim->impl;`	**Cast the generic optimizer implementation pointer to sgd_t.** The optimizer struct stores the concrete implementation as a void* to allow different optimizer types to use the same interface. The cast tells the compiler to treat those bytes as a sgd_t struct.
`for (size_t stateIndex = 0; stateIndex < optim->sizeStates; stateIndex++) {`	**Iterate over all trainable parameter pairs** (weights and biases).
`parameter_t *param = optim->parameter[stateIndex];`	**Get the i-th trainable parameter.**
`float *gradArr = (float *)param->grad->data;`	**Pointer to the gradient buffer for this parameter.**
`float *dataArr = (float *)param->param->data;`	**Pointer to the weight (parameter value) buffer.**
`for (size_t elementIndex = 0; elementIndex < numberOfValues; ++elementIndex) {`	**Update each individual weight element.**

`float grad = gradArr[elementIndex] + sgd->weightDecay * dataArr[elementIndex];`	**Effective gradient = accumulated gradient + weight decay term.** The weight decay term adds a small amount proportional to the current weight value, which will reduce the weight slightly regardless of the loss gradient. This prevents weights from growing very large over many training steps.
`dataArr[elementIndex] -= sgd->learningRate * grad;`	**Stochastic Gradient Descent update rule: w = w – learning_rate × gradient.** Subtracting moves the weight in the direction of decreasing loss. The learning rate scales how large each step is.

Stochastic Gradient Descent with Momentum (float code path):

`static void sgdStepMFloat(optimizer_t *optim) {`	**Stochastic Gradient Descent with Momentum step.**
`float grad = gradArr[elementIndex] + sgd->weightDecay * paramArr[elementIndex];`	**Compute the effective gradient with weight decay** — same as the vanilla version.
`stateArr[elementIndex] = sgd->momentumFactor * stateArr[elementIndex] + grad;`	**Velocity update: v = beta × v + gradient.** stateArr is the velocity buffer — it persists between training steps. beta (the momentumFactor) is typically 0.9, so the velocity retains 90% of its previous value and 10% is replaced by the new gradient. This exponentially weighted average of past gradients points in the consistent average direction of the gradient, filtering out the random variation between individual batches.
`paramArr[elementIndex] -= sgd->learningRate * stateArr[elementIndex];`	**Parameter update: w = w – learning_rate × velocity.** Momentum smooths out oscillations (particularly helpful in ravine-shaped loss surfaces where the gradient alternates direction across the narrow dimension) and can accelerate convergence by up to 1/(1–beta) times in consistent gradient directions.
`void sgdZeroGrad(optimizer_t *optimizer) {`	**Reset all gradient accumulators to zero.** Must be called after each optimizer step, before starting the next batch, to prevent gradients from accumulating across batches.
`memset(param->grad->data, 0, totalNumberOfBytes);`	**Fill the gradient buffer with zero bytes.** memset() is the C standard library function for filling a block of memory with a single byte value. Setting all bytes to 0 produces the integer 0 for every element, regardless of element type.
`if (param->grad->quantization->type == SYM_INT32) { symIntQ->scale = 1.f; }`	**Reset the Symmetric INT32 scale to 1.0.** After zeroing the integer gradient buffer, the scale factor from the previous batch is stale — it described how to interpret the old gradient values. Resetting to 1.0 ensures that when the first gradient of the new batch is stored, the scale is recomputed correctly from scratch.

Syntax Quick Reference

C constructs used across the codebase — written in full

C Language Constructs Used in This Codebase

Every C syntax construct that appears in the codebase is listed below with an example and a plain-English explanation. No abbreviations are used.

`#define SOURCE_FILE "FOO"`	Sets the file label used in debug macros. Must appear before #include "Common.h" in each .c file. The preprocessor replaces every occurrence of SOURCE_FILE in the code with the string "FOO".
`#ifndef / #define / #endif guard`	Include guard — prevents a header file from being processed more than once per compilation unit. Without it, duplicate type definitions would cause compiler errors.
`typedef enum { A, B } myEnum_t;`	Creates an enumeration type — a set of named integer constants. A = 0, B = 1, and so on by default. The typedef makes the name usable without writing "enum" every time.
`typedef struct foo { int x; } foo_t;`	Creates a named struct type accessible as foo_t. A struct groups several variables together under one name. Each variable inside is called a member or field.
`typedef union bar { int* a; float* b; } bar_t;`	All members of a union share the same memory address. Only one member is valid at a time. An external "discriminator" field (usually an enum) tells you which member is currently active.
`typedef void (*fn_t)(int a);`	Creates a function-pointer type. fn_t is now the name for "pointer to a function that takes one int and returns nothing". Function pointers enable runtime dispatch — storing different functions in an array and calling them through the pointer.
`[LINEAR] = {fnA, fnB}`	C99 designated initialiser syntax. Initialises a specific array element (at index LINEAR) by name instead of by position. Makes vtable definitions readable because you can see which layer type each row corresponds to.
`static void helper(void) {}`	Restricts the function (or variable) to the current .c file. Functions defined with "static" are invisible to other translation units — this is C's equivalent of "private" in object-oriented languages.
`void* qConfig;`	Generic pointer — can point to any type. Must be cast to the specific type before dereferencing. Used in quantization_t so the same struct can hold any of five different config struct types.
`memcpy(&x;, bytes, 4);`	The only C standard-conforming way to reinterpret a byte sequence as a typed value. Avoids undefined behaviour from pointer aliasing (which would occur if you cast a byte pointer to a float pointer and read through it).
`do { stmt; } while(false)`	Wraps multiple statements into a single expression that requires a semicolon, making it safe to use after an unbraced if-statement. Used in all macro definitions. The loop body runs exactly once.
`printf(fmt, ##__VA_ARGS__)`	GNU extension: if no variadic arguments are passed to the macro, the ## (double hash) before __VA_ARGS__ removes the comma that would otherwise appear before the empty argument list, preventing a syntax error.

`float buf[n]; tensor_t t;`	Variable-Length Array on the stack (valid since C99). The size n can be a runtime value, not just a compile-time constant. Used extensively to avoid heap allocations in hot paths on the Microcontroller Unit.
`extern layerFunctions_t layerFunctions[];`	Declaration: the array is defined in exactly one .c file; all other files that include this header share access to the same array. Prevents "multiple definition" linker errors while allowing read/write access from any file.
`size_t`	The standard unsigned integer type for sizes, counts, and array indices. It is guaranteed to be large enough to hold any object size on the current platform (32-bit on 32-bit systems, 64-bit on 64-bit systems).
`uint8_t / int32_t / uint32_t`	Fixed-width integer types from stdint.h. uint8_t is an unsigned 8-bit integer (range 0–255). int32_t is a signed 32-bit integer (range −2,147,483,648 to +2,147,483,647). uint32_t is an unsigned 32-bit integer (range 0 to 4,294,967,295). These types have the same size on every platform, unlike plain "int" which varies.
`const float input`	"const" before a parameter type is a promise from the function that it will not modify the value. This documents intent, enables compiler optimisations, and prevents accidental mutation.

Next step — Guide 3: Phases 9–13 (DataLoader, Serialization, InferenceApi, TrainingLoopApi, MnistExperiment) with the same line-by-line treatment. Guides 1 and 2 together give complete coverage of the foundation, memory management, tensor system, arithmetic, Random Number Generator, layer system, loss functions, and optimizer subsystems.